

Noise, Hardware, and Reality

Haimeng Zhang
Quantum Algorithm Engineer

Contents

Part one

- 1 Coherence
- 2 Decoherence
- 3 Readout errors
- 4 Gate errors

Part two

- 5 Algorithm constraints
- 6 Error suppression, error mitigation and error correction

Part three

- 7 Run circuits on simulators vs. real backends
- 8 Interpreting error-prone results

Part four

- 9 Noise characterization mini-lab
- 10 Introduce a simple error suppression technique
- 11 Discussion and reflection

Contents

Part one

- 1 Coherence
- 2 Decoherence
- 3 Readout errors
- 4 Gate errors

Part two

- 5 Algorithm constraints
- 6 Error suppression, error mitigation and error correction

Part three

- 7 Run circuits on simulators vs. real backends
- 8 Interpreting error-prone results

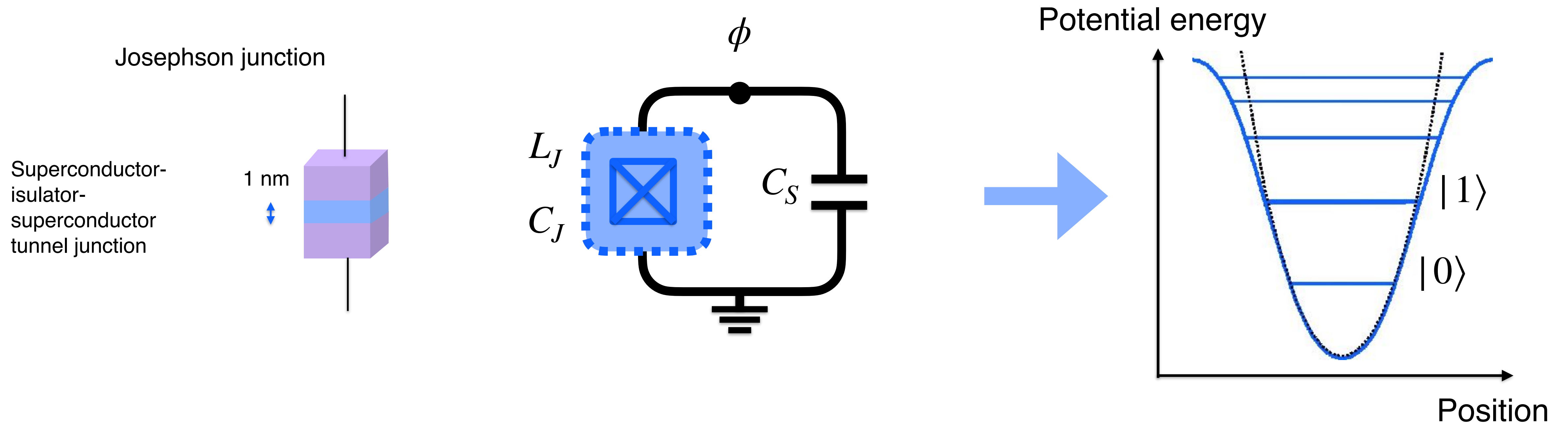
Part four

- 9 Noise characterization mini-lab
- 10 Introduce a simple error suppression technique
- 11 Discussion and reflection

Coherence

Coherence is the ability of a qubit to maintain a well-defined quantum state over time.

Quantization in an electric circuit



- A superconducting qubit can be viewed as an LC oscillator — the Josephson junction acts as a nonlinear inductor;
- The phase ϕ across the junction is a macroscopic quantum variable;
- With anharmonicity in the potential energy, we can isolate a two-level system and drive $|0\rangle \leftrightarrow |1\rangle$ without exciting higher levels;
- Coherence is the ability to maintain a quantum superposition state $\alpha|0\rangle + \beta|1\rangle$.

Decoherence

Quantum information is fragile

Examples of noise in superconducting hardware:

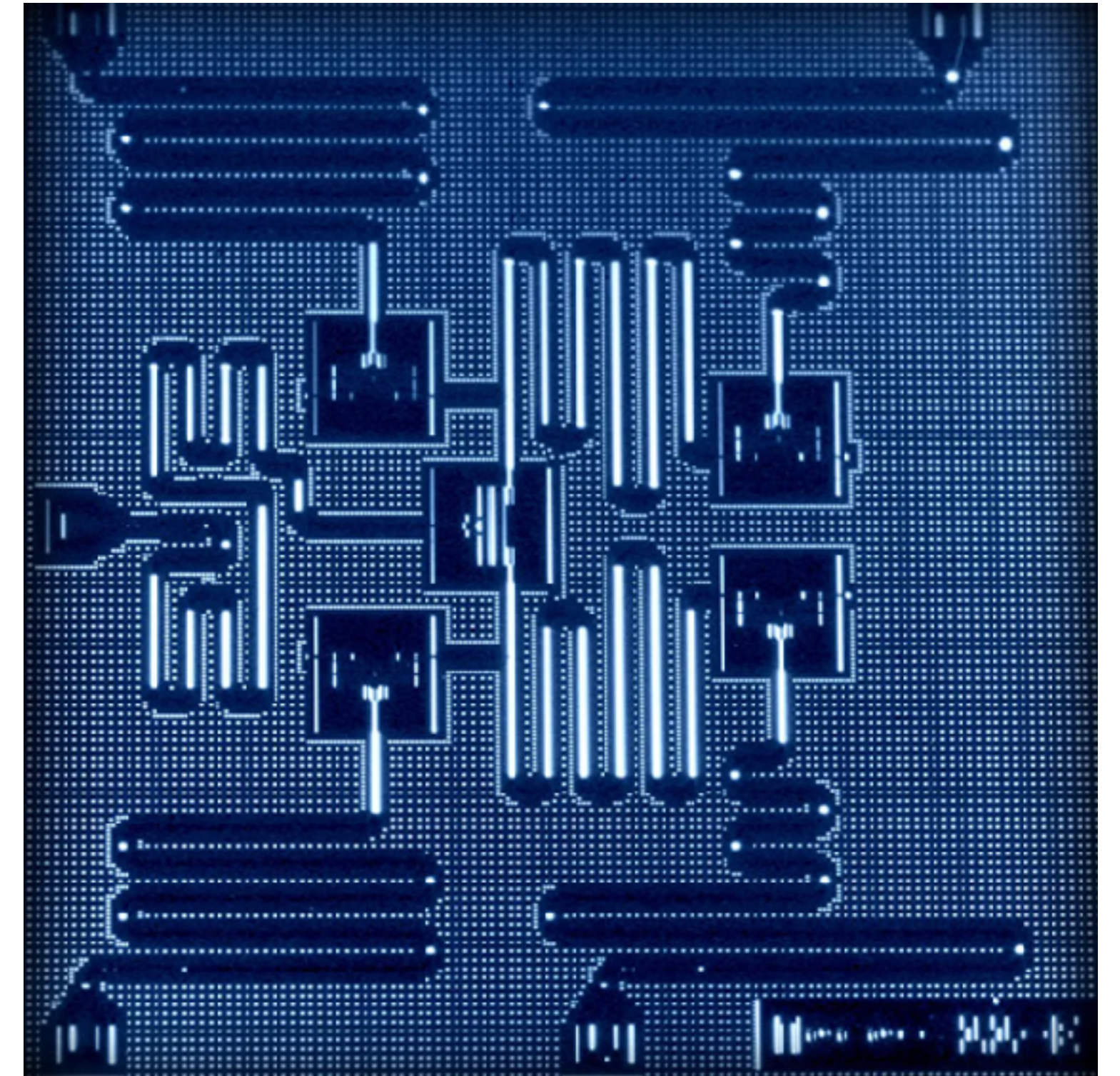
Charge noise: fluctuating electric fields from two-level systems in the substrate cause qubit frequency drift and qubit dephasing

Flux noise: magnetic flux fluctuation cause qubit frequency shift

Dielectric loss: microwave energy is absorbed by the material defects in the Josephson junction or substrate, shorten qubit T1 time

Quasiparticle poisoning: broken Cooper pairs tunnel across the junction, causing sudden bit-flip-like errors

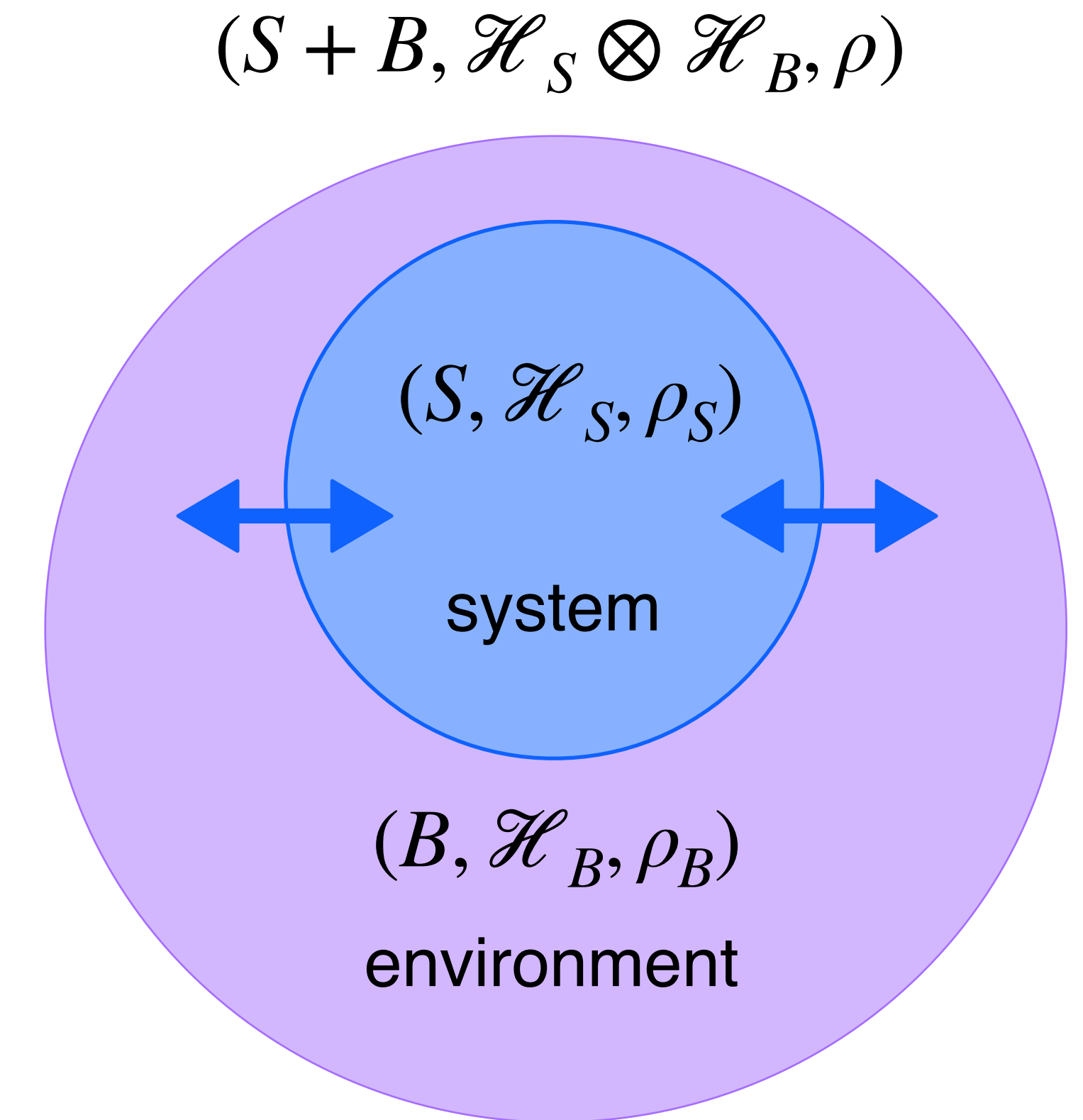
Crosstalk: stray interaction between neighbouring qubits



Decoherence

Decoherence is a result of system interacting with its environment.

	Closed system	Open system
State	$ \psi_S\rangle \in \mathbb{C}^{2^n}$ State vector in $\mathcal{H}_S$ Pure state $\rho_S = \psi_S\rangle\langle\psi_S $	$\rho_S \in \mathbb{C}^{2^n \times 2^n}$ Density matrix Mixed state $\rho_S = \sum_i p_S \psi_S^{(i)}\rangle\langle\psi_S^{(i)} $
Evolution	Unitary U Schrödinger Equation	Complete positive trace preserving map $\mathcal{L}$ Quantum master equation e.g., in Lindblad form



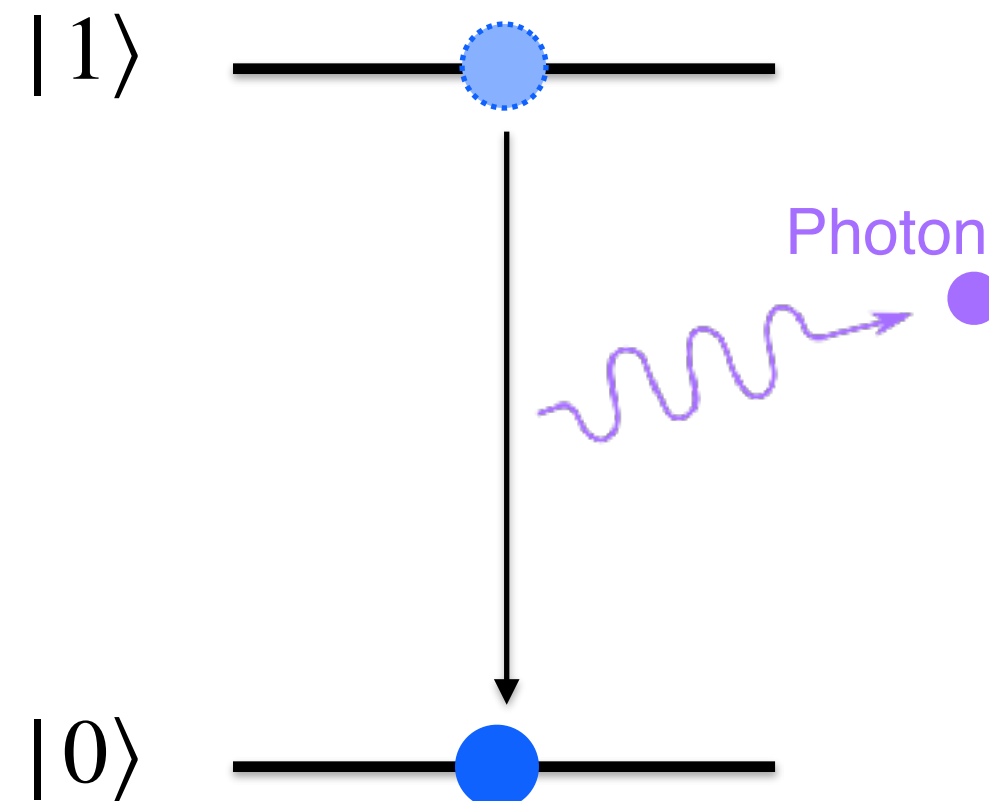
To learn more about density matrices, see courses by John Watrous, [General formulation of quantum information](#)

Decoherence

T_1 relaxation process

What is T_1 relaxation process?

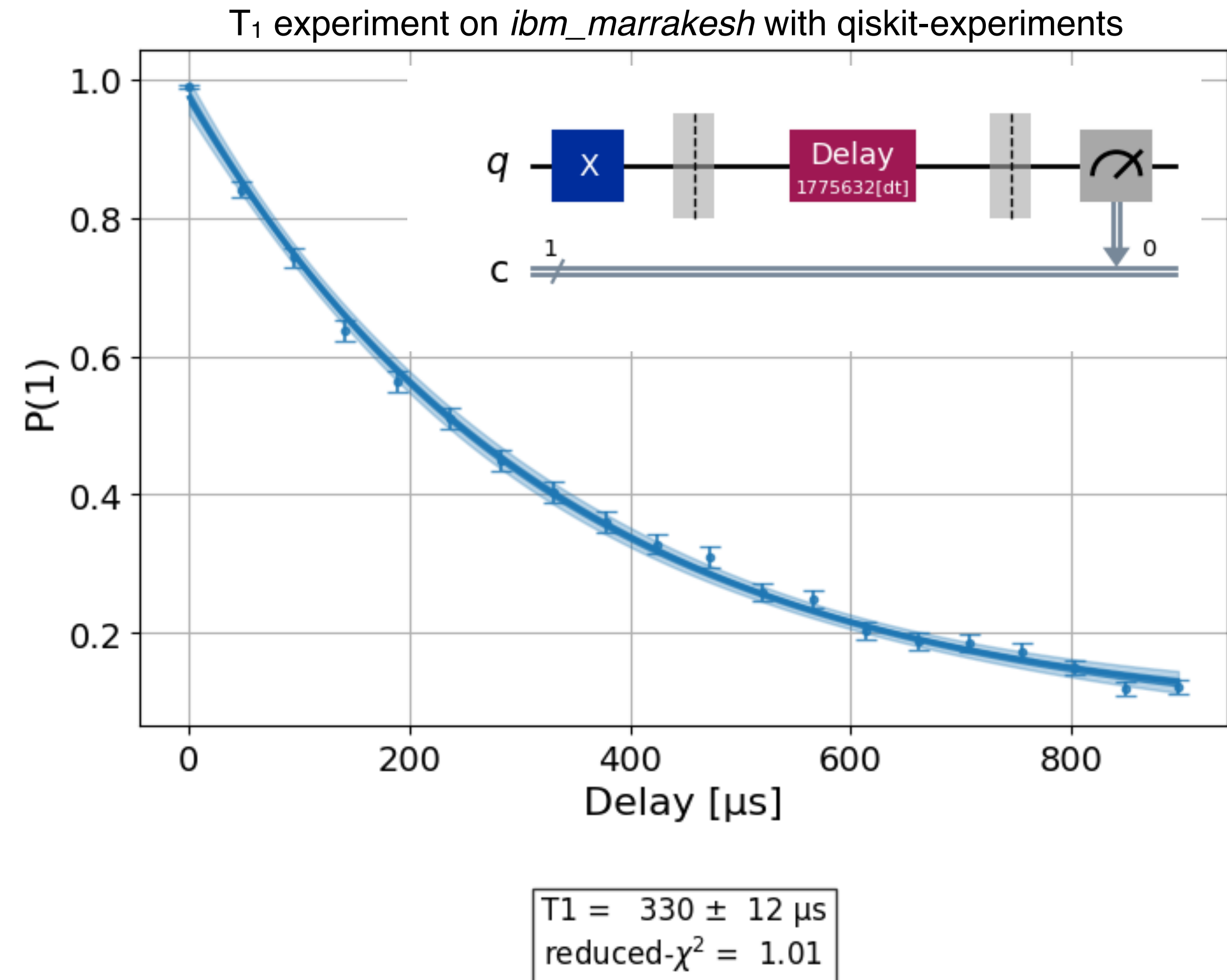
A process by which a qubit spontaneously loses its excitation energy to the environment, causing it to decay from the excited state $|1\rangle$ to the ground state $|0\rangle$.



Spontaneous emission: a consequence of the coupling of the system to electromagnetic fields.

The probability of the state remains in $|1\rangle$ decays exponentially

$$P_1(t) = e^{-t/T_1}$$



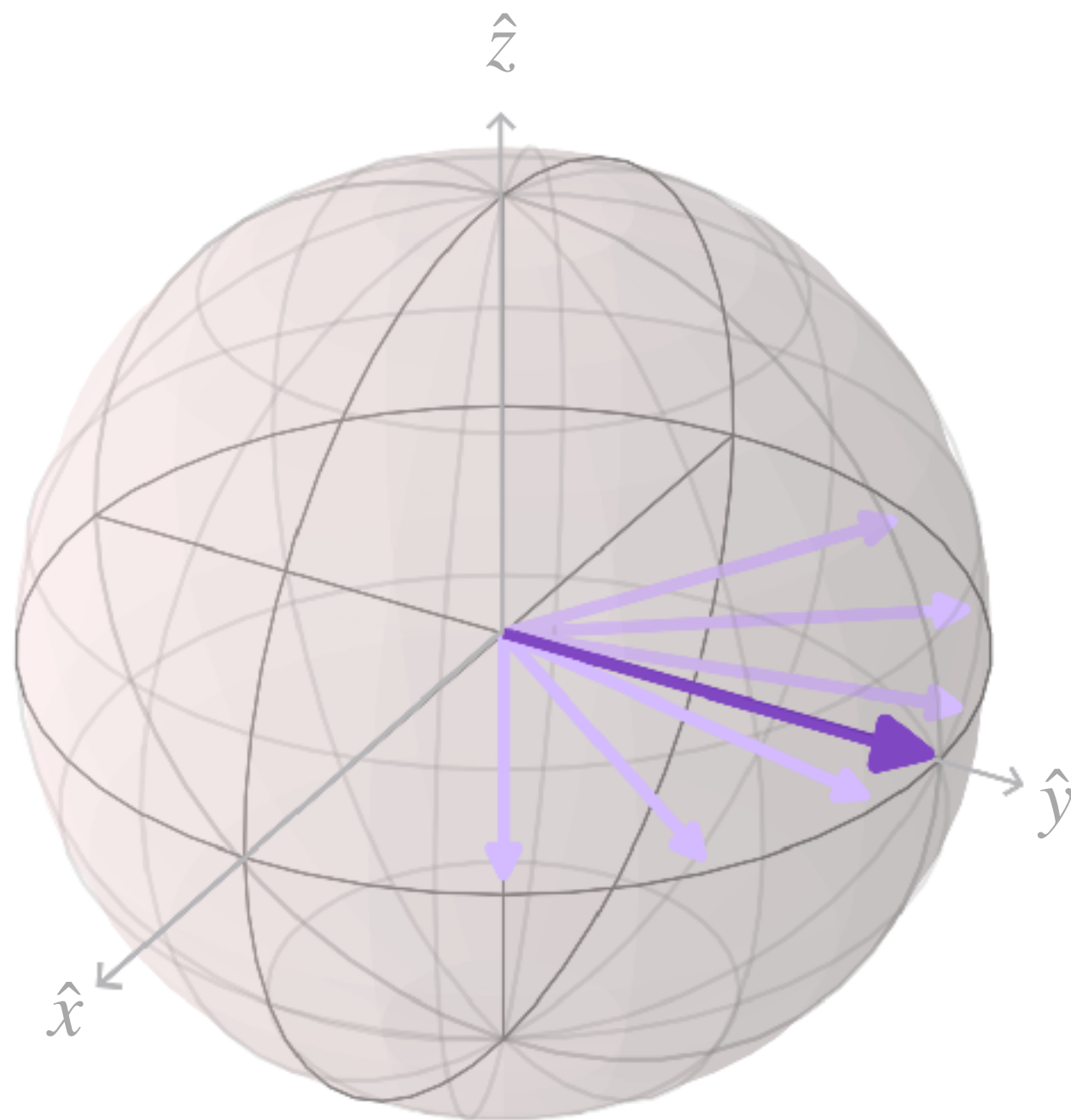
- After one T_1 period, about 37% of the excitation population remains. After three T_1 s, less than 5%.
- A gate on the backend takes roughly 50-100 ns, if $T_1 = 100 \mu s$, we get on the order of 1000-5000 gates before T_1 errors become significant.

Decoherence

T₂ dephasing process

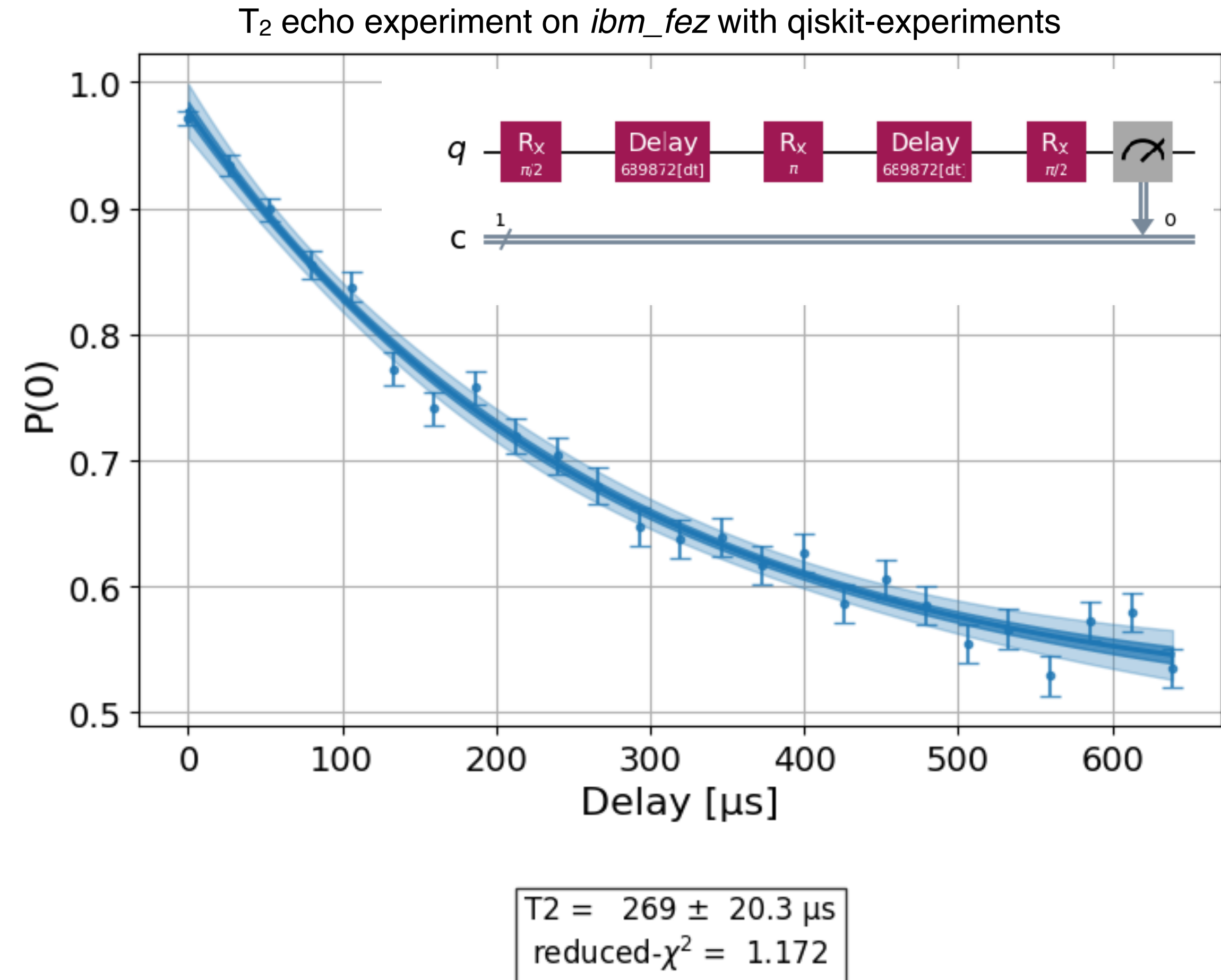
What is T₂ dephasing process?

A process by which a qubit loses its coherent phase information, caused by fluctuations in the qubit transition frequency.



The qubit coherence (off-diagonal elements of the density matrix) decays exponentially:

$$\langle \sigma_x(t) \rangle = e^{-t/T_2}$$



The two contributions to T₂: $\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}$

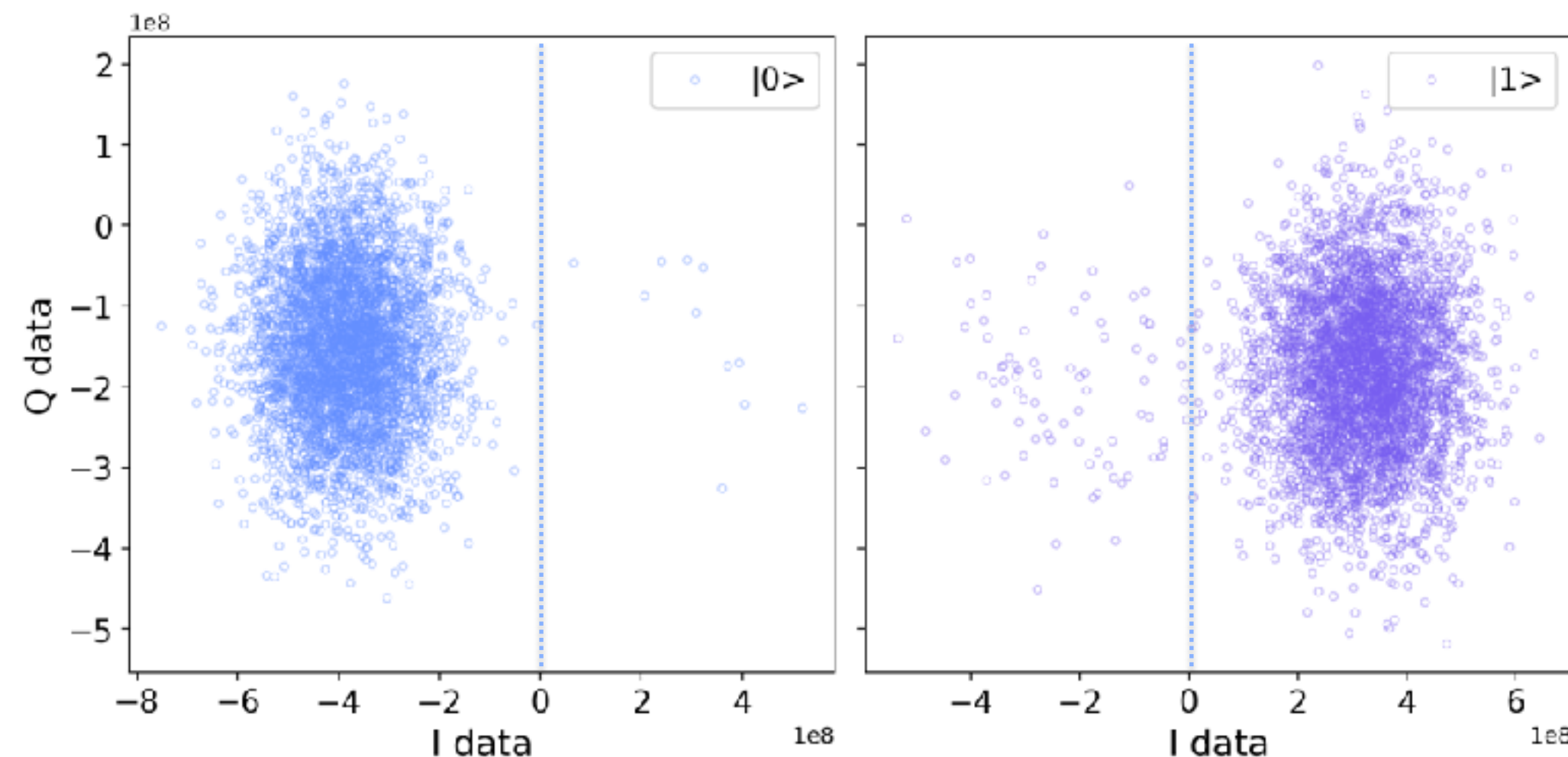
Readout errors

In the qubit readout process, a qubit is entangled with its readout resonator via the dispersive interaction, and the qubit state is read out via an amplified microwave signal from the readout resonator.

I (In-phase) and Q (Quadrature) are the two orthogonal components of a microwave signal. Any microwave signal can be expressed as:

$$s(t) = I \cdot \cos(2\pi f t) - Q \cdot \sin(2\pi f t)$$

Initialize qubit state in $|0\rangle$ or $|1\rangle$ and measure on ibm_fez



To classify a shot as 0 or 1, one can place a decision boundary between the two blobs (e.g., a line or using a Gaussian classifier). The readout fidelity depends on:

- $p(1 | 0)$: probability of reporting 1 when qubit is in $|0\rangle$
- $p(0 | 1)$: probability of reporting 0 when qubit is in $|1\rangle$

A 2^n by 2^n matrix (M) is constructed to represent the probability of observing state i given true state j .

If the ideal bitstring distribution is described by vector $\vec{p}_{\text{true}}$, the measured distribution is:

$$\vec{p}_{\text{meas}} = M \vec{p}_{\text{true}}$$

Two-qubit confusion matrix

$$M = \begin{pmatrix} P(00|00) & P(00|01) & P(00|10) & P(00|11) \\ P(01|00) & P(01|01) & P(01|10) & P(01|11) \\ P(10|00) & P(10|01) & P(10|10) & P(10|11) \\ P(11|00) & P(11|01) & P(11|10) & P(11|11) \end{pmatrix}$$

Gate errors

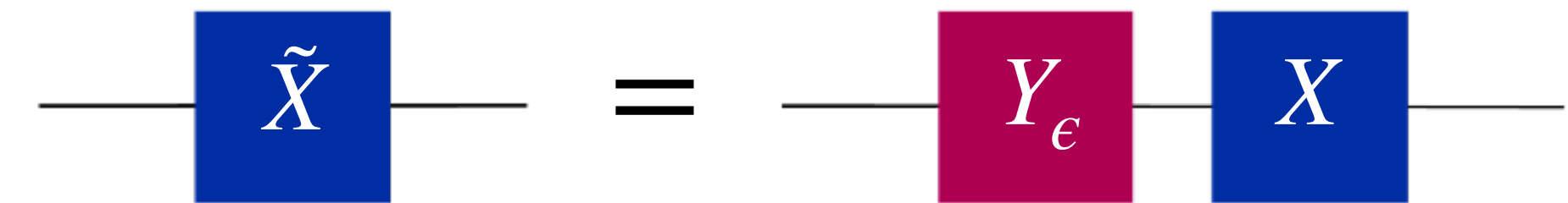
Gate errors can include:

1. Coherent/systematic error: deterministic miscalibrations, e.g., over-rotation
2. Incoherent errors: due to decoherence and relaxation
3. Crosstalk due to unwanted inter-qubit couplings

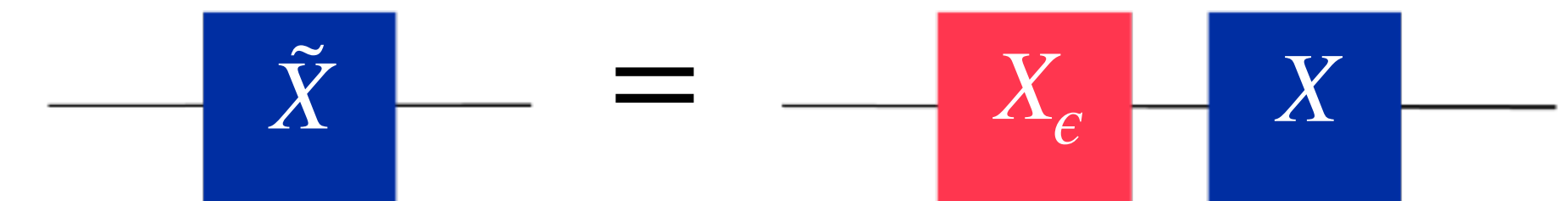
Coherent errors are worth characterizing because they are correctable, whereas incoherent errors are fundamentally stochastic and irreversible.

Consider $X(\pi)$ gate

Misaligned rotation



Over/under rotation



Contents

	Part one
1	Coherence
2	Decoherence
3	Readout errors
4	Gate errors
	Part two
5	Algorithm constraints
6	Error suppression, error mitigation and error correction
	Part three
7	Run circuits on simulators vs. real backends
8	Interpreting error-prone results
	Part four
9	Noise characterization mini-lab
10	Introduce a simple error suppression technique
11	Discussion and reflection

Algorithm constraints

Device connectivity

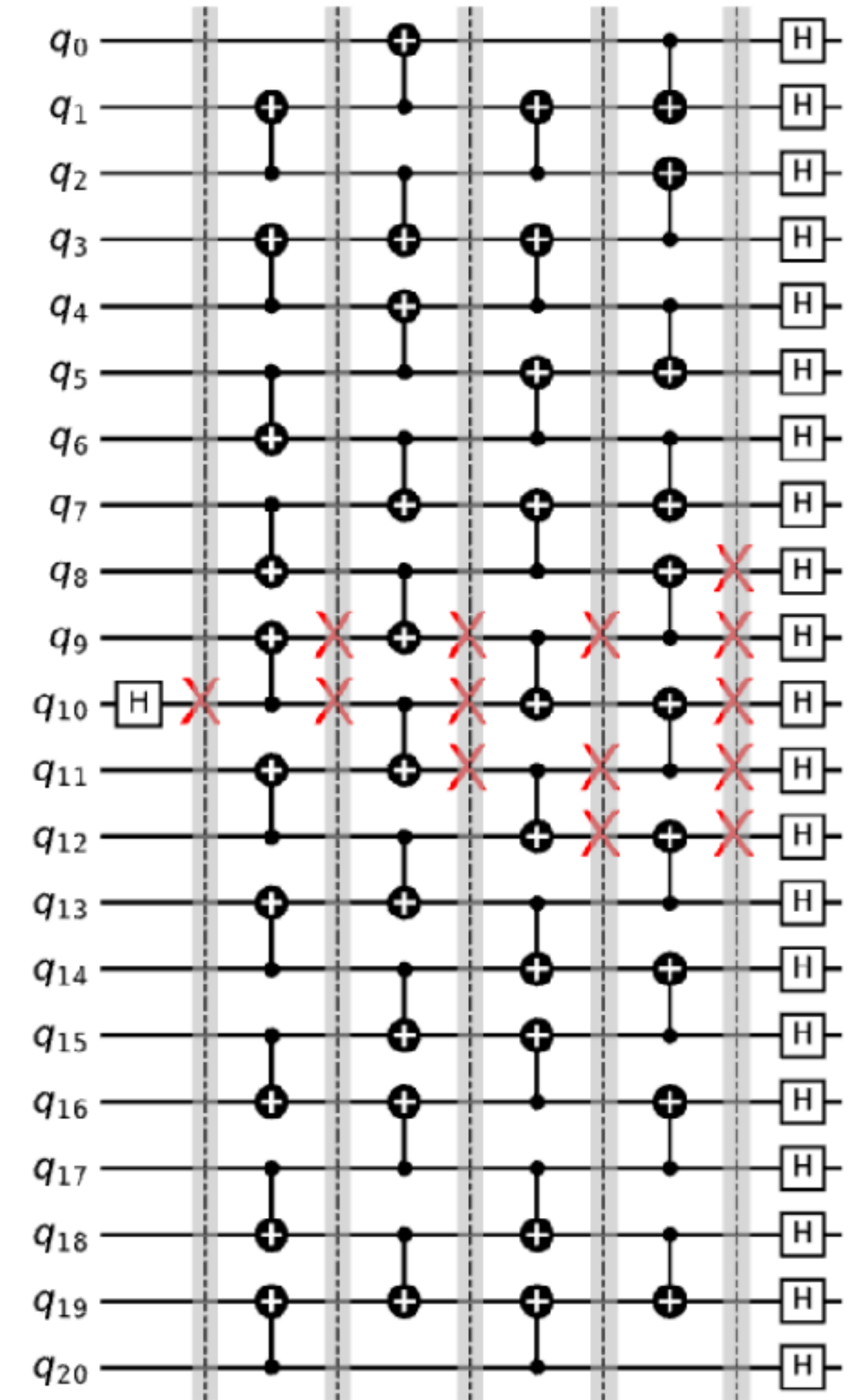
- Fixed device connectivity (heavy-hex or square lattice) introduces overhead from adding SWAP gates for qubit routing, increasing circuit depth and circuit errors;
- Limited device connectivity justifies a quasi-local noise assumption, reducing the number of characterization parameters to scale linearly with qubit count — keeping noise learning tractable on near-term devices.

Circuit depth

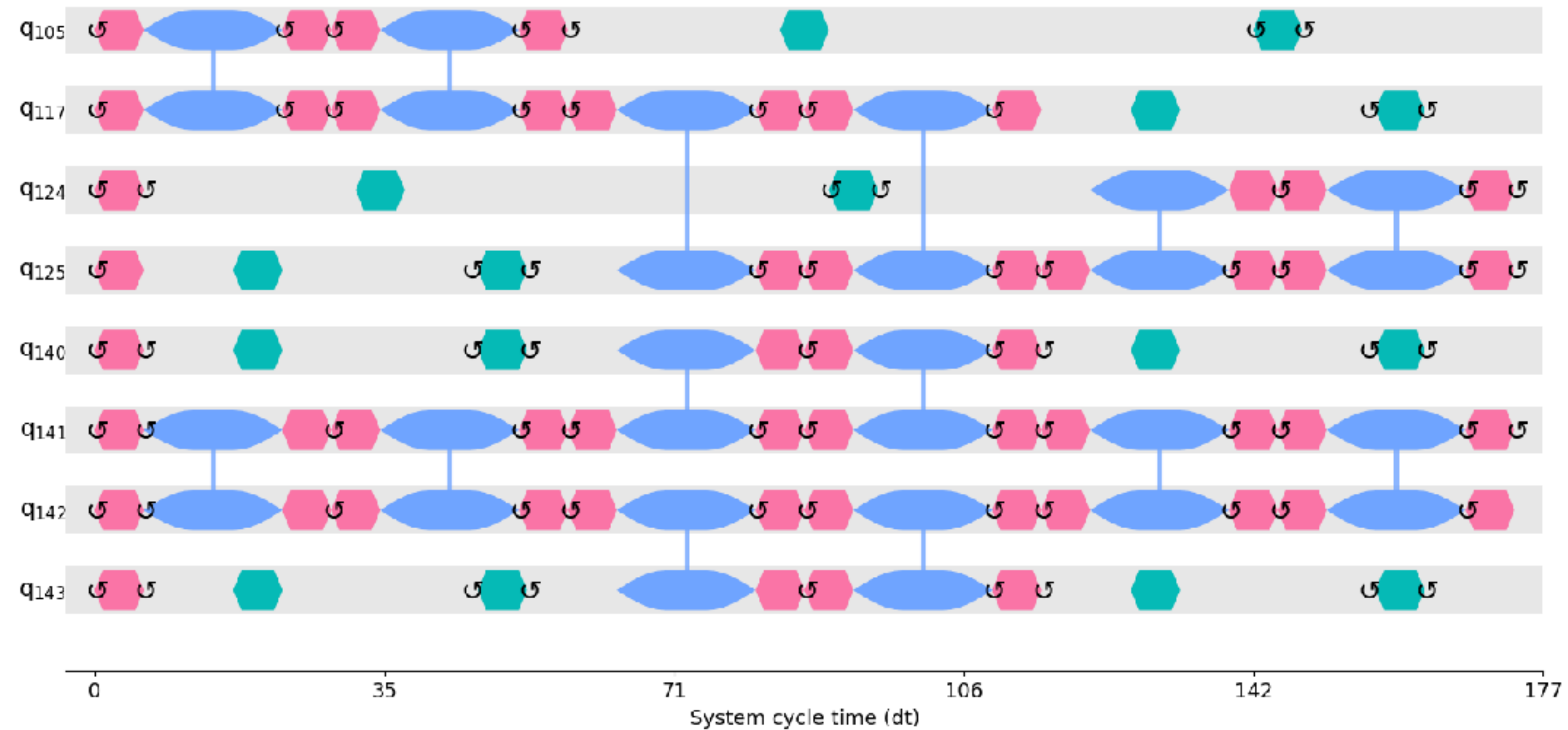
- Circuit fidelity degrades with depth, ultimately limited by T_1/T_2 coherence times ($\sim 50\text{--}100\ \mu\text{s}$, or $\sim 100\text{--}300$ two-qubit gate layers);
- Circuit compression and approximation (e.g., aqc-tensor, an approximate circuit compression method) are introduced to trade accuracy for hardware executability.

Qubit counts

- Error propagates through entangling gates across qubits;
- Circuit width is bounded by where the algorithm's output remains distinguishable from the noise floor.



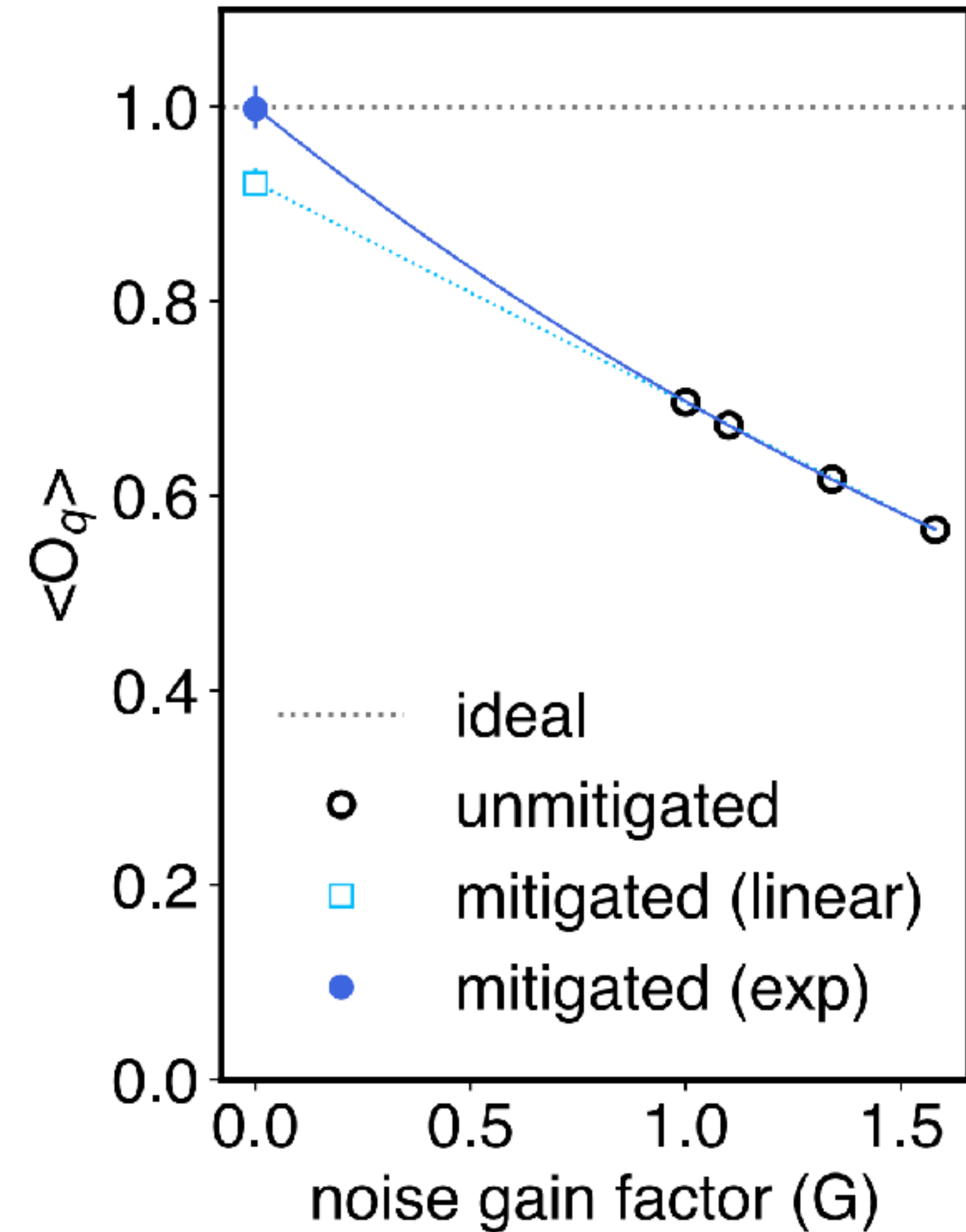
Error suppression



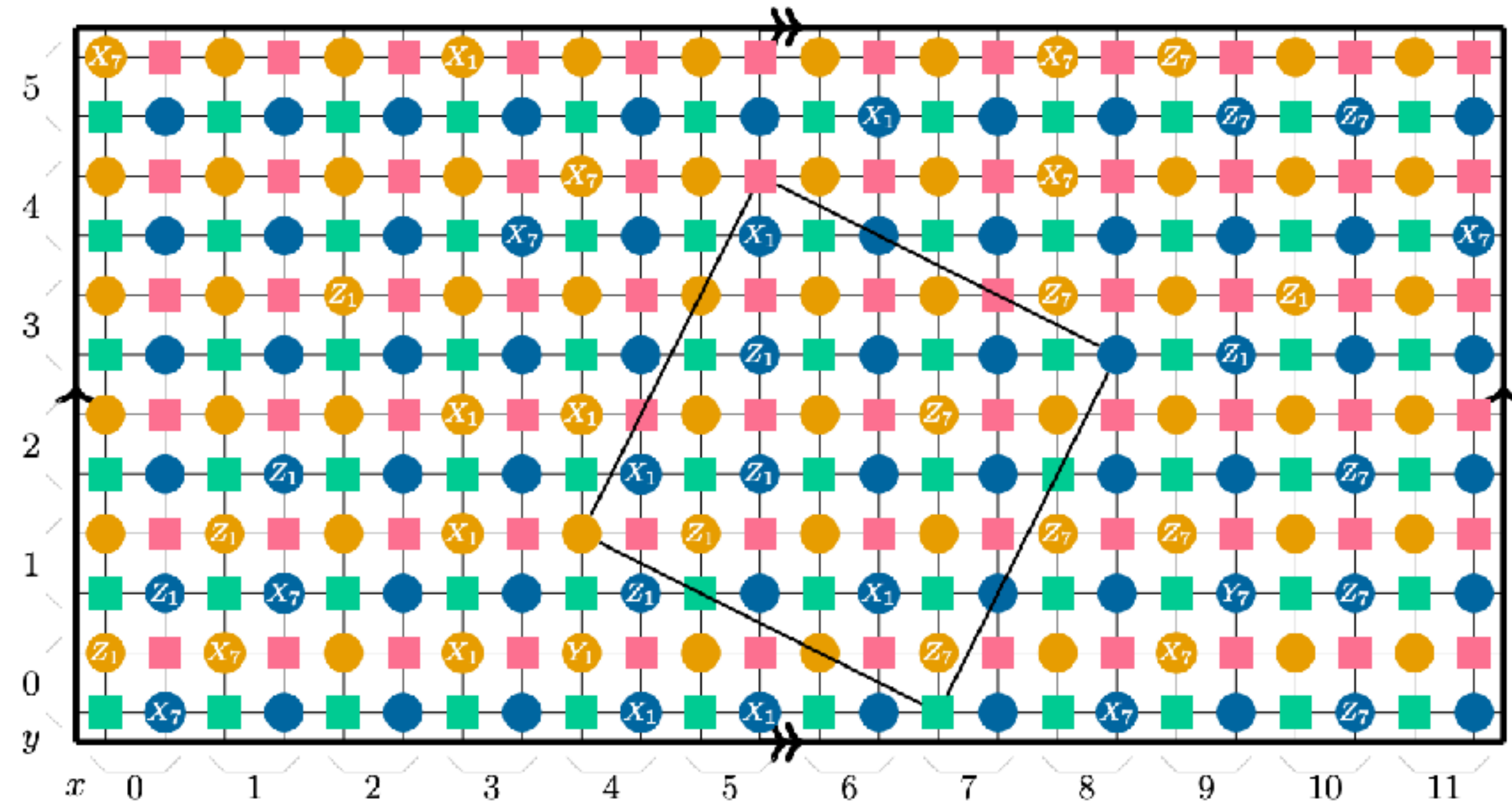
- Actively reduces error rates during computation by designing better pulses or gate compilation sequences;
- Examples: dynamical decoupling, optimal control pulses, noise-aware compilation;
- Typically minimal runtime overhead on the quantum device.

Error mitigation

- Estimates the noise-free expectation value by post-processing noisy measurement results;
- Examples: zero-noise extrapolation [1], probabilistic error cancellation;
- ZNE requires running the circuit at multiple noise levels and extrapolates to the zero-noise limit;
- PEC has exponential shot overhead that scales as $e^{2\gamma d}$, where γ is the noise rate and d is the circuit depth, also requires additional overhead to learn device noise model.



Error correction



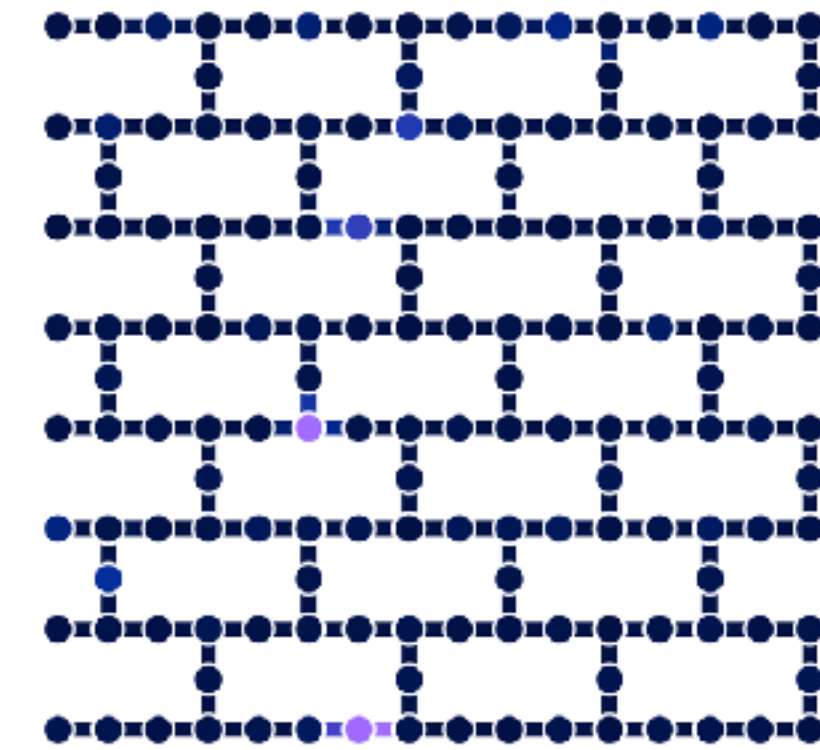
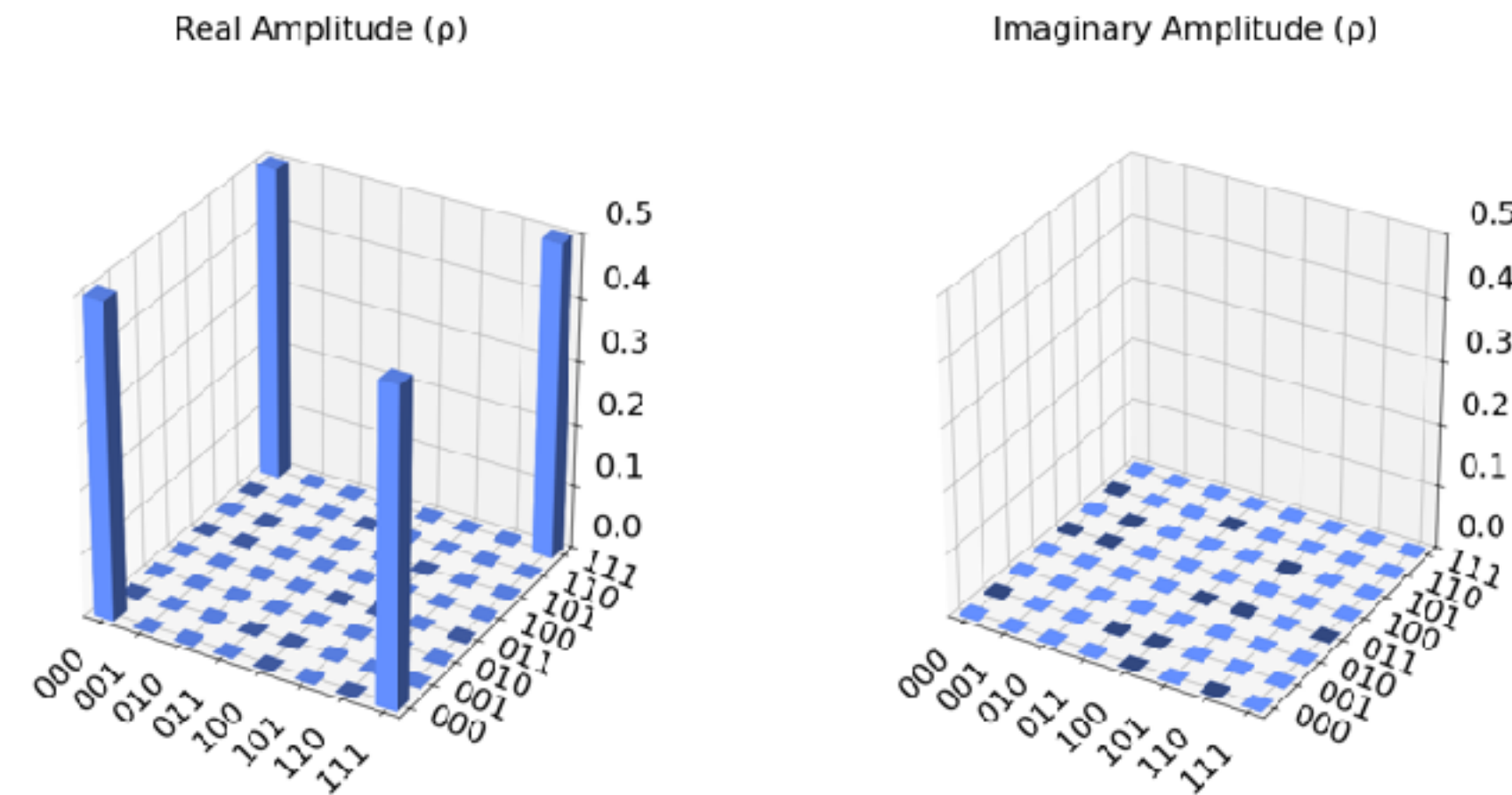
- Encodes logical qubits in entangled physical qubits and actively detects and corrects errors;
- Stabilizer codes: detect and correct errors via syndrome measurements;
- Low-density parity check (LDPC) codes [1]: more resource-efficient than surface codes asymptotically;
- Requires additional ancilla qubits for encoding per logical qubit, mid-circuit measurement and classical feedforward;
- Universal gate sets require magic state distillation for non-Clifford operations.

Contents

	Part one
1	Coherence
2	Decoherence
3	Readout errors
4	Gate errors
	Part two
5	Algorithm constraints
6	Error suppression, error mitigation and error correction
	Part three
7	Run circuits on simulators vs. real backends
8	Interpreting error-prone results
	Part four
9	Noise characterization mini-lab
10	Introduce a simple error suppression technique
11	Discussion and reflection

Run circuits on simulators vs. real backends

$$|\psi\rangle = \begin{bmatrix} \alpha_{000} \\ \alpha_{001} \\ \alpha_{010} \\ \alpha_{011} \\ \alpha_{100} \\ \alpha_{101} \\ \alpha_{110} \\ \alpha_{111} \end{bmatrix}$$



Ideal (noiseless) simulator

- Quantum states are stored as state vectors of size 2^n
- A 50-qubit system requires ~8 PB of RAM. Exact wave function simulation is infeasible beyond ~30 qubits on classical hardware.
- System symmetries can reduce memory usage but cannot overcome the fundamental exponential scaling.
- Best for algorithm development and verification at small qubit counts

Noisy simulator (qiskit-aer)

- Quantum states are represented by density matrices to capture mixed states and decoherence.
- Applies noise channels (e.g., depolarizing, T_1 / T_2 and readout errors) after each gate to mimic hardware behaviors.
- Density matrix representation scales as 4^n , more memory intensive than ideal simulators.
- Ideal for testing algorithm robustness to noise and error mitigation strategies.

Real backend

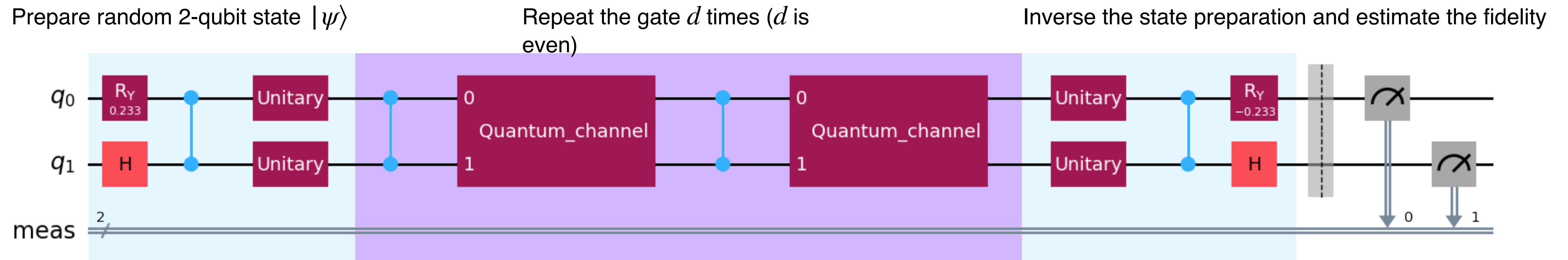
- Information encoded in qubits
- Noise is real and complex — it varies across qubits and drifts over time.
- Scalability of algorithms is hardware-constrained; qubit count, connectivity, and gate fidelities are fixed by the device.
- Best for experimentally validate quantum algorithms at scale and for problems where quantum advantage over classical simulation is the goal.

Run circuits on simulators vs. real backends

How to amplify noise for better characterization?

Run a gate G repeatedly for d times and measure at the end, how does the accumulated error grow with d ?

Consider a benchmarking circuit



- The fidelity F is estimated from measuring P_{00} , averaged over an ensemble of initial states;
- For ideal circuits, $F=1$;
- A Quantum channel contains the description about noise; we consider depolarizing noise and coherent noise;
- Readout noise is also considered in the measurement step.

Interpreting error-prone results

Depolarizing noise

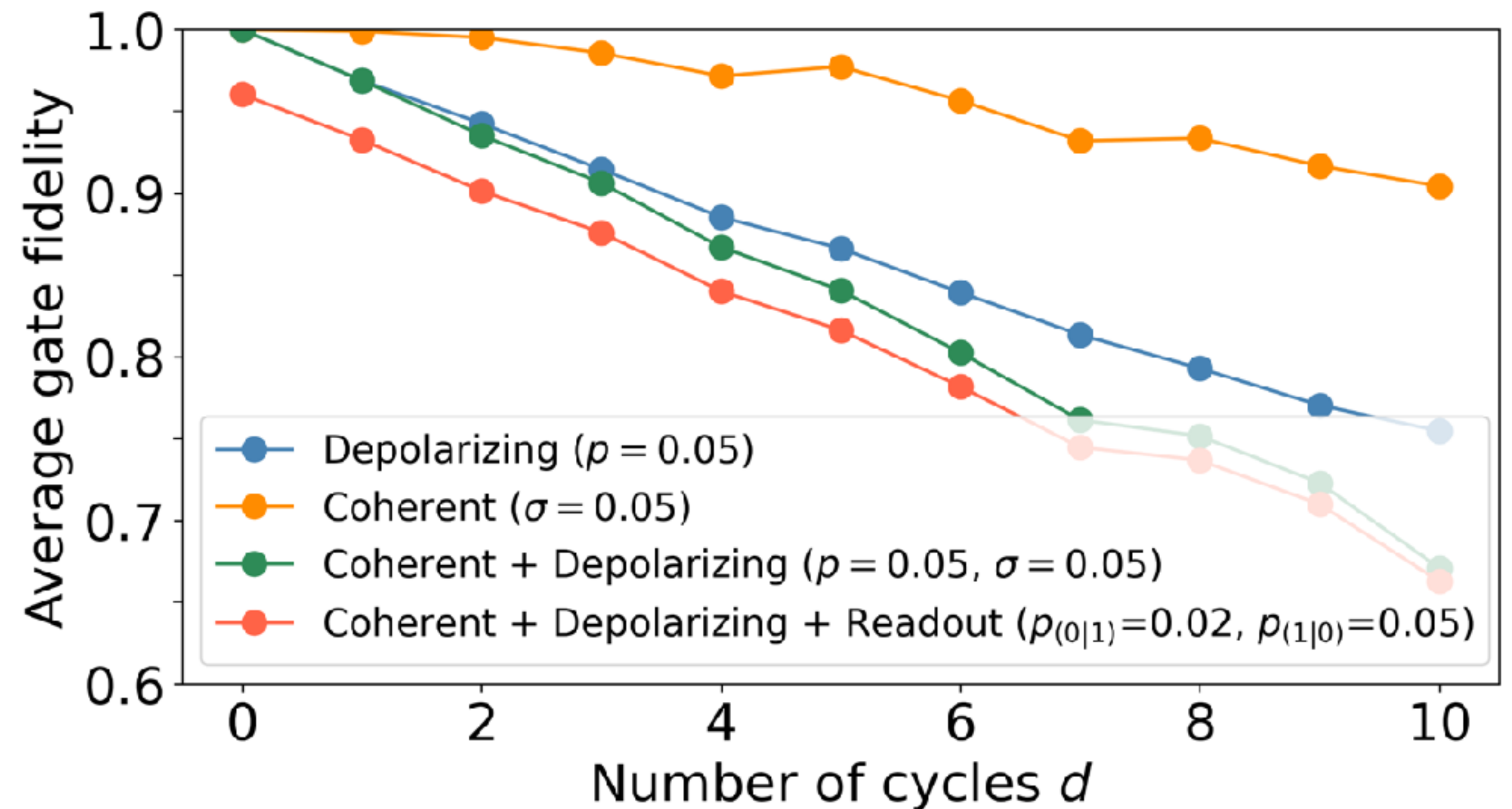
$$\mathcal{D}(\rho) = (1 - p)\rho + p\frac{I}{2^n}$$

Coherent noise:

$$U(\theta) = \text{CPhase}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{pmatrix}, \quad \theta \sim \mathcal{N}(0, \sigma^2)$$

Readout error:

$$M = \begin{pmatrix} 1 - p(1|0) & p(1|0) \\ p(0|1) & 1 - p(0|1) \end{pmatrix}$$



Coherent error accumulates quadratically with d in short time limit
 Incoherent error accumulates linearly with d in short time limit
 SPAM errors introduce a constant offset that is independent of d

Contents

	Part one
1	Coherence
2	Decoherence
3	Readout errors
4	Gate errors
	Part two
5	Algorithm constraints
6	Error suppression, error mitigation and error correction
	Part three
7	Run circuits on simulators vs. real backends
8	Interpreting error-prone results
	Part four
9	Noise characterization mini-lab
10	Introduce a simple error suppression technique
11	Discussion and reflection

Noise characterization mini lab

Gate to characterize: XXPlusYYGate(θ , β)

$$R_{XX+YY}(\theta, \beta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\left(\frac{\theta}{2}\right) & -i \sin\left(\frac{\theta}{2}\right) e^{-i\beta} & 0 \\ 0 & -i \sin\left(\frac{\theta}{2}\right) e^{i\beta} & \cos\left(\frac{\theta}{2}\right) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- The gate introduces a parameterized rotation in the one-excitation subspace, mixing $|01\rangle$ and $|10\rangle$ states while leaving $|00\rangle$ and $|11\rangle$ unchanged.
- This gate is particle-number preserving: it cannot create or destroy excitations, only move them between qubits. It appears, e.g., when simulating spin exchange in the Heisenberg model or constructing fermionic orbital rotations for quantum chemistry.
- When executed on real hardware, it is transpiled to two CZ gates, with the remaining parameters absorbed into single-qubit rotations.

Noise characterization mini lab

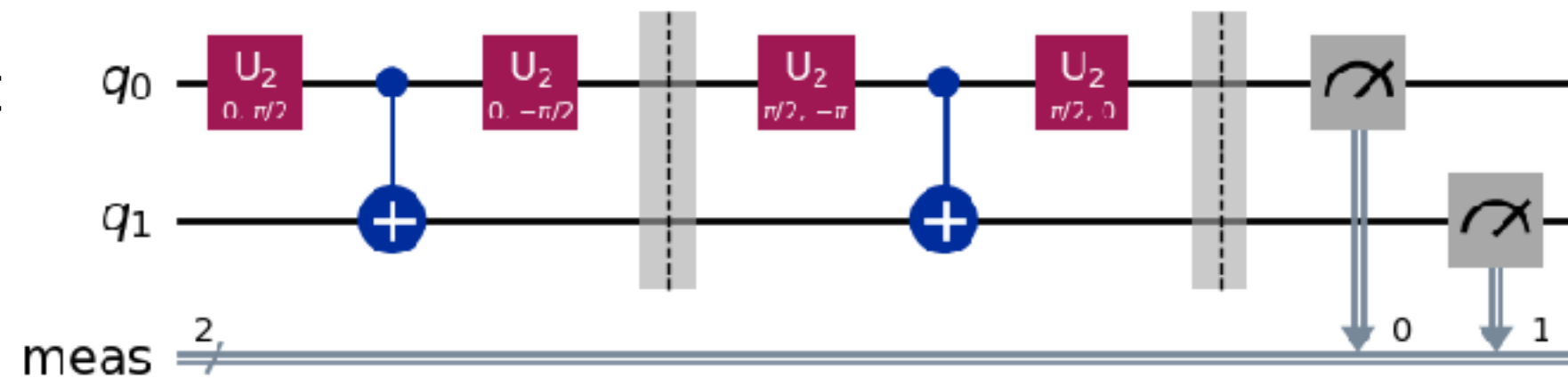
Randomized benchmarking

A method for characterizing the average error rate of quantum gate operations by applying random sequences of gates and measuring how quickly the output fidelity decays.

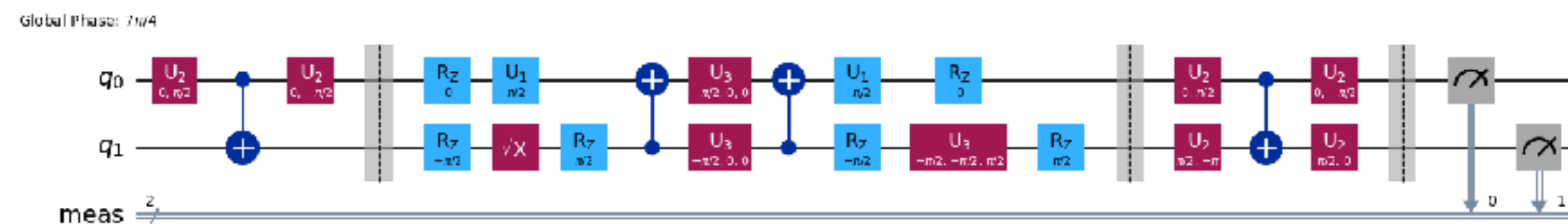
Run two types of random Clifford circuits:

- a reference sequence
- an interleaved sequence where the target gate is inserted between every random Clifford.

Reference circuit



Interleaved circuit



The analysis outputs the gate fidelity and EPC (error per Clifford)

```
import numpy as np
from qiskit.circuit import QuantumCircuit
from qiskit.circuit.library import XXPlusYYGate
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit_experiments.library import InterleavedRB
from qiskit_ibm_runtime import QiskitRuntimeService

# --- 1. Connect and pick backend ---
service = QiskitRuntimeService()
backend = service.least_busy(min_num_qubits=2)
physical_qubits = (0, 1)

# --- 2. Build and transpile the gate under test ---
qc = QuantumCircuit(2)
qc.append(XXPlusYYGate(theta=np.pi, beta=0.0), (0, 1))

pm = generate_preset_pass_manager(
    optimization_level=3,
    basis_gates=backend.basis_gates,
)
transpiled_gate = pm.run(qc)

# --- 3. Run Interleaved RB ---
irb = InterleavedRB(
    interleaved_element=transpiled_gate,
    physical_qubits=physical_qubits,
    lengths=[1, 5, 10, 20, 50],
    backend=backend,
    num_samples=20,
    seed=42,
)
exp_data = irb.run().block_for_results()

# --- 4. Results ---
display(exp_data.figure(0))
display(exp_data.analysis_results(dataframe=True))
```

python

Noise characterization mini lab

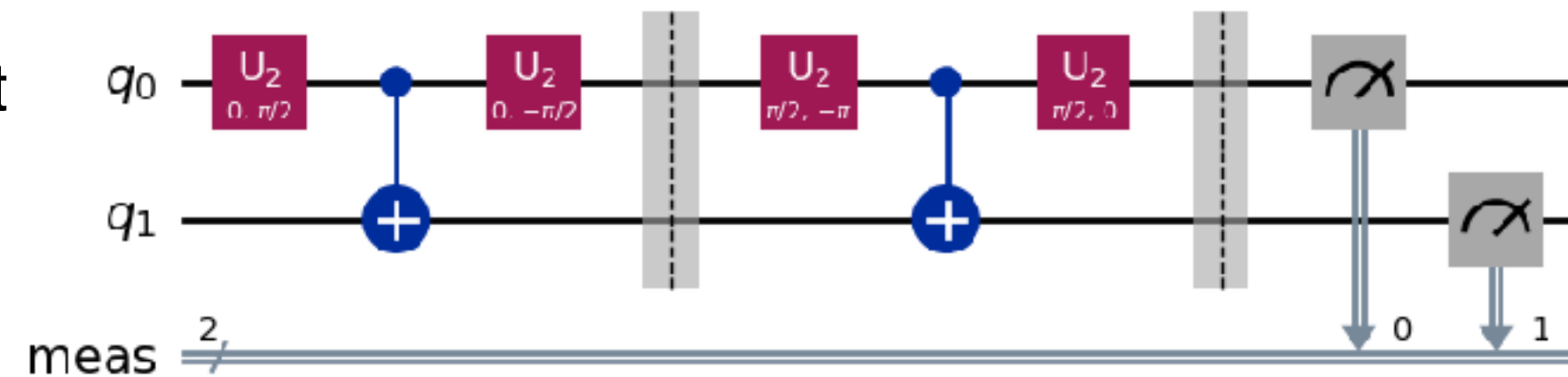
Randomized benchmarking

A method for characterizing the average error rate of quantum gate operations by applying random sequences of gates and measuring how quickly the output fidelity decays.

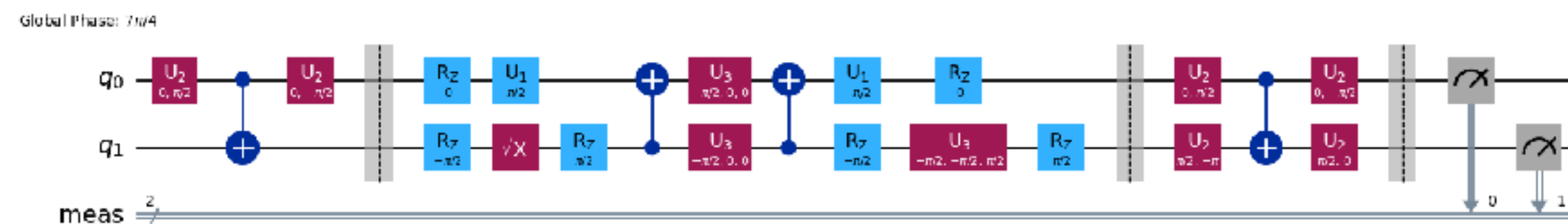
Run two types of random Clifford circuits:

- a reference sequence
- an interleaved sequence where the target gate is inserted between every random Clifford.

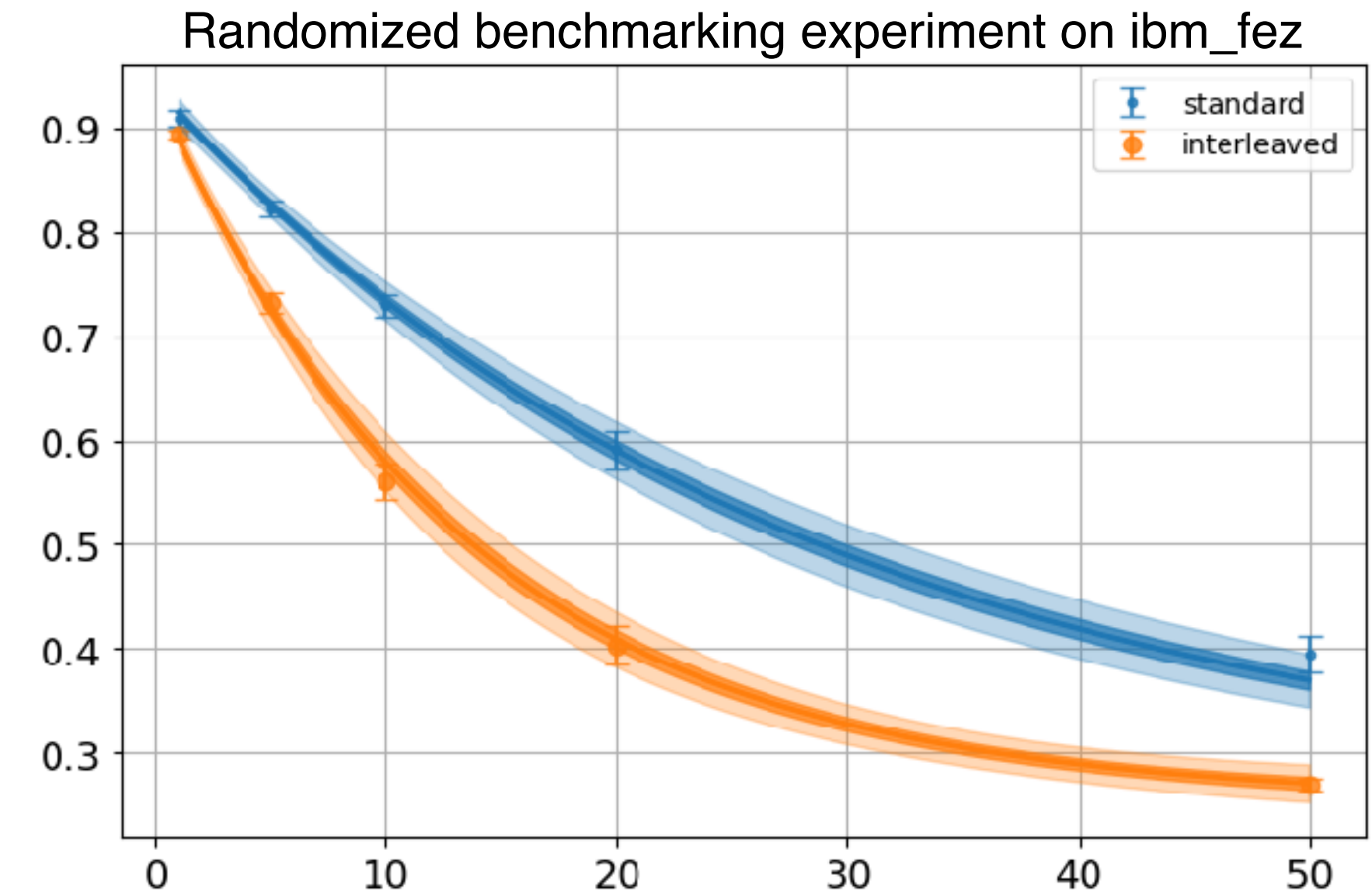
Reference circuit



Interleaved circuit



The analysis outputs the gate fidelity and EPC (error per Clifford)



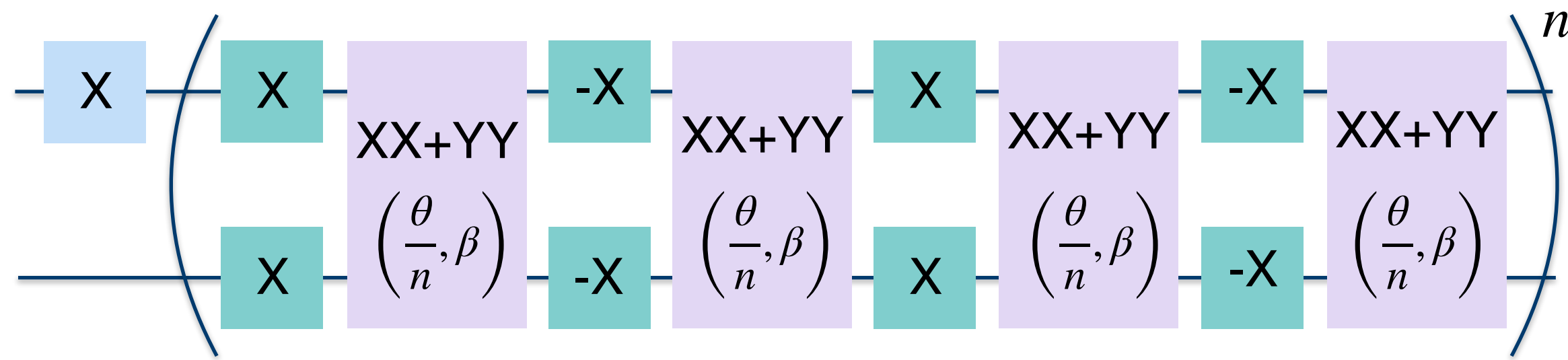
$\alpha = 0.965 \pm 0.001671$
 $\alpha_c = 0.9618 \pm 0.003327$
 $\text{EPC} = 0.02865 \pm 0.002496$
 $\text{reduced-}\chi^2 = 0.9308$

$\text{EPC} = (1 - r_{\text{interleaved}} / r_{\text{reference}}) \times (D-1)/D$,
 where r is the decay rate and D is the Hilbert space dimension

Noise characterization mini lab

Introduce an error suppression technique: dynamical decoupling

DD can be used on a gate-level [1]: X gates commute with XX+YY gates, and anti-commute with noise operator Z.



1. Trotterize XX+YY gate into d equal steps, $\text{XXPlusYY}(\theta) \approx [\text{XXPlusYY}(\theta/n)]^n$
2. For each trotter step, two versions of the circuits are built: no DD and with DD
3. Perform state tomography at each trotter step and compute the state tomography with respect to the ideal state

• State fidelity: $F(\rho_1, \rho_2) = \text{Tr} \left[\sqrt{\sqrt{\rho_1} \rho_2 \sqrt{\rho_1}} \right]^2$, when ρ_1 is a pure state,
 $\rho_1 = |\psi_1\rangle\langle\psi_1|$, we have $F(\rho_1, \rho_2) = \langle\psi_1|\rho_2|\psi_1\rangle$

[1] Gross, J. A. et al., *npj Quantum Information* **10**, 123 (2024)

```
import numpy as np
from qiskit.circuit import QuantumCircuit
from qiskit.circuit.library import XXPlusYYGate
from qiskit.quantum_info import DensityMatrix, state_fidelity
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit_experiments.library import StateTomography
from qiskit_experiments.framework import BatchExperiment

TOTAL_THETA = np.pi / 2 + 2 * np.pi
TROTTER_STEPS = [4 * n for n in range(1, 20)]
PHYSICAL_QUBITS = (24, 25)

qc_ideal = QuantumCircuit(2)
qc_ideal.x(0)
qc_ideal.append(XXPlusYYGate(theta=TOTAL_THETA, beta=0), (0, 1))
dm_ideal = DensityMatrix(qc_ideal)

backend = QiskitRuntimeService().least_busy()
pm = generate_preset_pass_manager(backend=backend, optimization_level=3)

def Xm(qc, q):
    """X- gate: conjugate X by Rz(n) to flip the rotation axis"""
    qc.rz(np.pi, q)
    qc.x(q)
    qc.rz(-np.pi, q)

def make_trotter(n):
    qc = QuantumCircuit(2)
    qc.x(0)
    for _ in range(n):
        qc.append(XXPlusYYGate(TOTAL_THETA / n, 0), (0, 1))
    return pm.run(qc)

def make_trotter_dd(n):
    qc = QuantumCircuit(2)
    qc.x(0)
    for _ in range(n // 4):
        for _ in range(2):
            # F- steps: conjugate with X-
            for q in (0, 1): Xm(qc, q)
            qc.append(XXPlusYYGate(TOTAL_THETA / n, 0), (0, 1))
        for _ in range(2):
            # F+ steps: conjugate with X
            for q in (0, 1): qc.x(q)
            qc.append(XXPlusYYGate(TOTAL_THETA / n, 0), (0, 1))
    return pm.run(qc)

exps = [StateTomography(f(n), physical_qubits=PHYSICAL_QUBITS)
        for n in TROTTER_STEPS for f in (make_trotter, make_trotter_dd)]

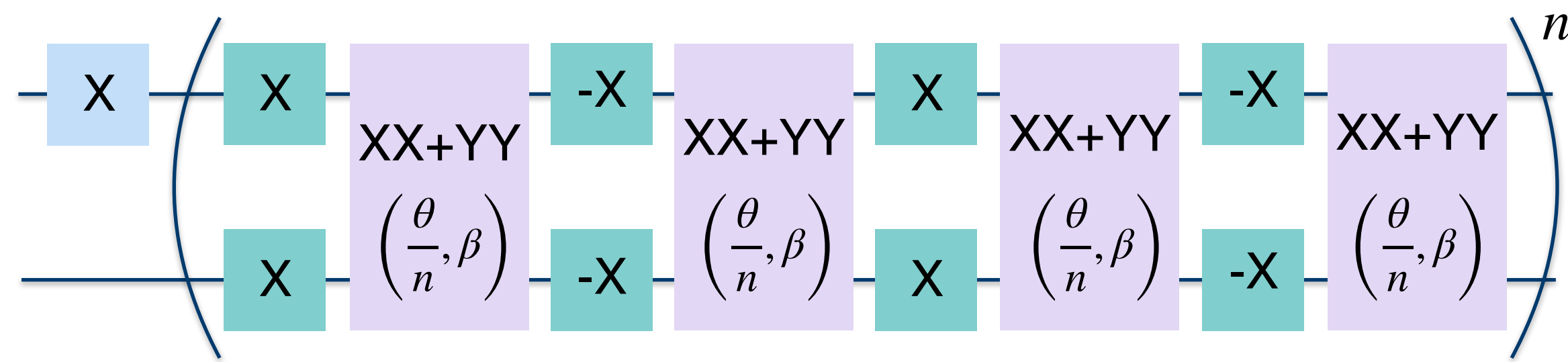
batch_data = BatchExperiment(exps, flatten_results=False).run(backend, shots=8192).block_for_results()

fidelities, fidelities_dd = [], []
for i in range(len(TROTTER_STEPS)):
    for lst, idx in [(fidelities, 2*i), (fidelities_dd, 2*i+1)]:
        dm = batch_data.child_data(idx).analysis_results("state", dataframe=True)["value"].iloc[0]
        lst.append(state_fidelity(dm, dm_ideal))
```


Noise characterization mini lab

Introduce an error suppression technique: dynamical decoupling

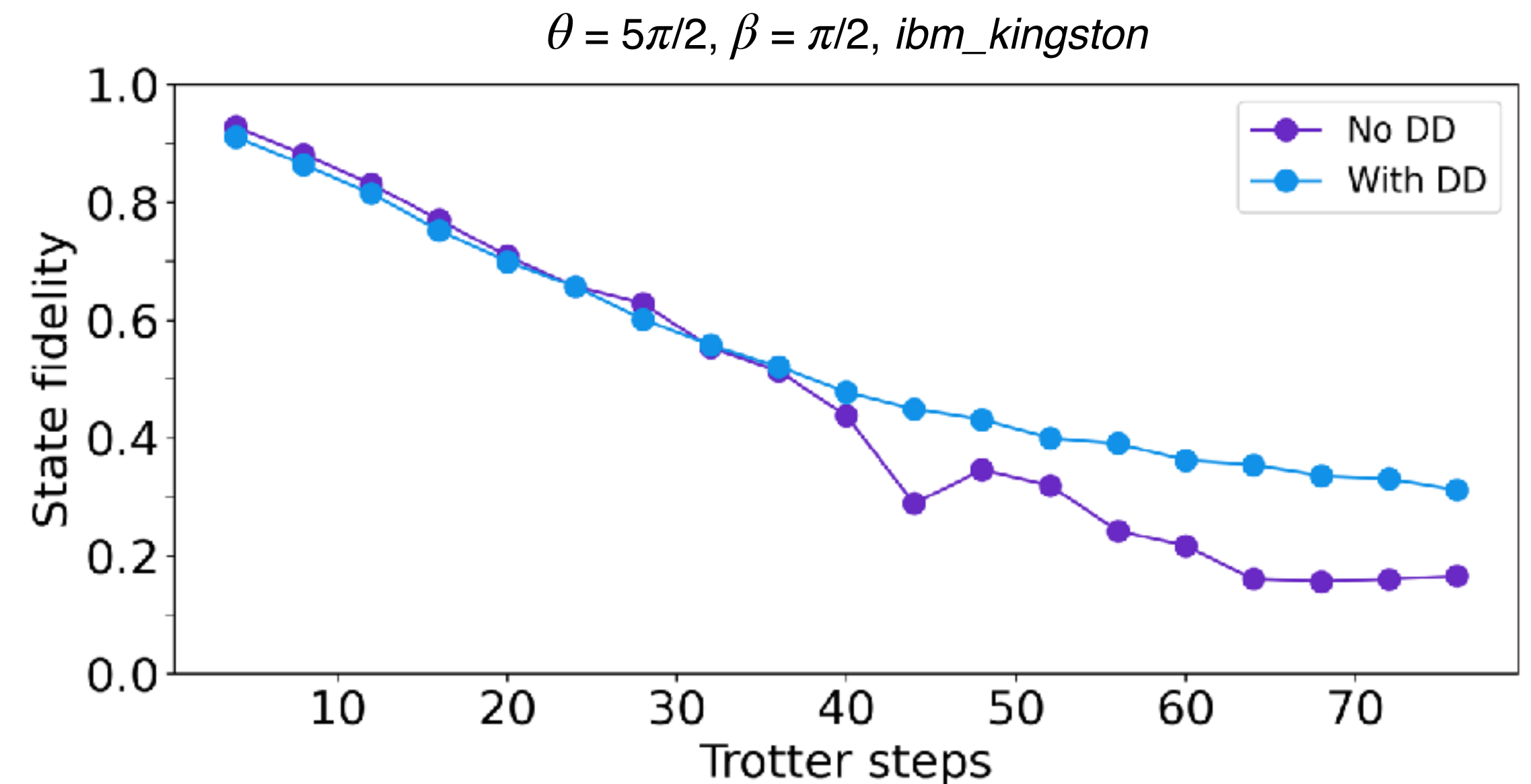
DD can be used on a gate-level [1]: X gates commute with XX+YY gates, and anti-commute with noise operator Z.



1. Trotterize XX+YY gate into d equal steps, $XXPlusYY(\theta) \approx [XXPlusYY(\theta/n)]^n$
2. For each trotter step, two versions of the circuits are built: no DD and with DD
3. Perform state tomography at each trotter step and compute the state tomography with respect to the ideal state

• State fidelity: $F(\rho_1, \rho_2) = \text{Tr} \left[\sqrt{\sqrt{\rho_1} \rho_2 \sqrt{\rho_1}} \right]^2$, when ρ_1 is a pure state,
 $\rho_1 = |\psi_1\rangle\langle\psi_1|$, we have $F(\rho_1, \rho_2) = \langle\psi_1|\rho_2|\psi_1\rangle$

[1] Gross, J. A. et al., *npj Quantum Information* **10**, 123 (2024)



DD is more effective at large trotter steps where gate errors accumulate; at short trotter step, adding DD can hurt fidelity due to additional gates that DD introduces.

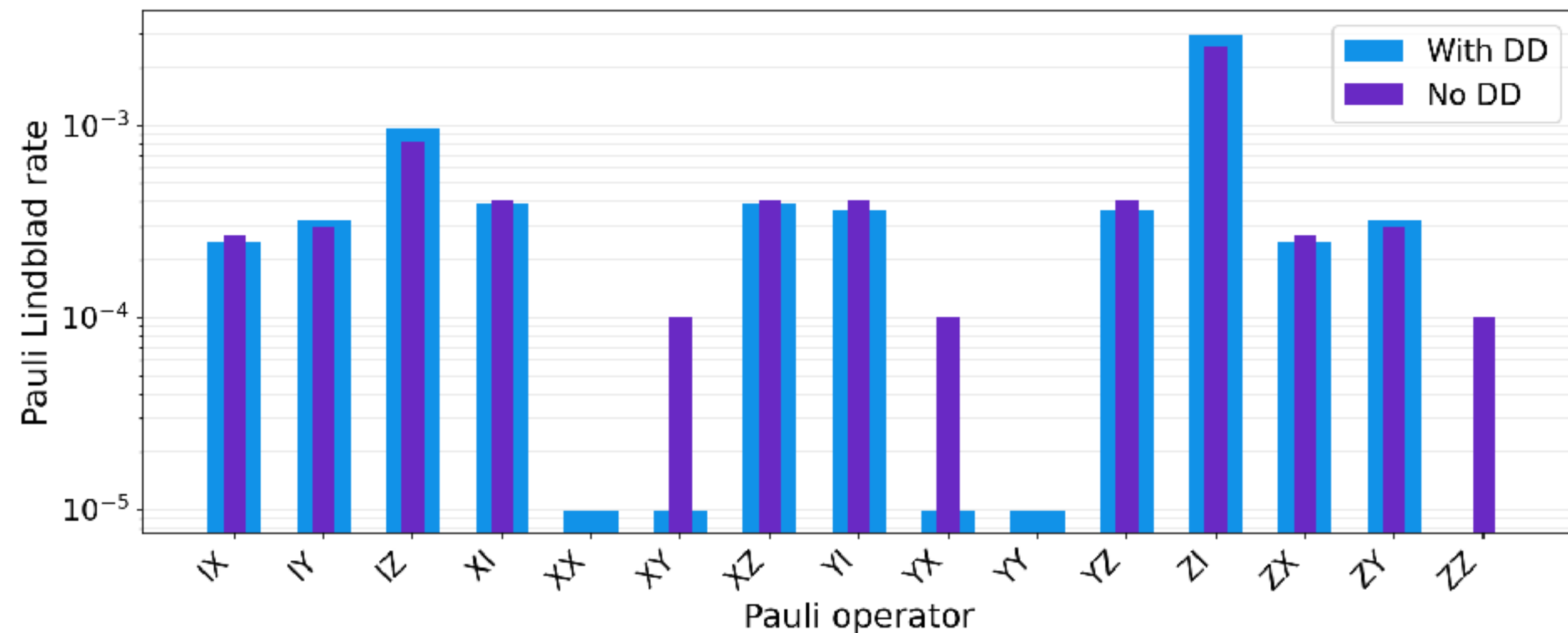
Noise characterization mini lab

Noise learner to examine the structure of the noise

Pauli-Lindblad noise model

$$\Lambda(\rho) = \exp[\mathcal{L}](\rho) = \prod_{k \in \mathcal{K}} \left(w_k \cdot \mathbb{I} + (1 - w_k) P_k \cdot P_k^\dagger \right) \rho$$

Generator:
$$\mathcal{L}(\rho) = \sum_{k \in \mathcal{K}} \lambda_k \left(P_k \rho P_k^\dagger - \rho \right)$$



- DD suppresses errors XY, YX and ZZ errors significantly;
- DD does not uniformly suppress all noise — ZI and IZ rates barely change;
- Some channels (XX, YY) actually increase with DD — DD introduces its own gate error overhead.

```
import numpy as np
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit_ibm_runtime.noise_learner import NoiseLearner
from qiskit_ibm_runtime.options import NoiseLearnerOptions

# --- Backend ---
service = QiskitRuntimeService()
backend = service.least_busy()

# --- Build / transpile circuits ---

# --- NoiseLearner options ---
options = NoiseLearnerOptions()
options.layer_pair_depths = [1, 3, 5, 7, 9, 11] # cycle depths for Pauli noise learning
options.max_layers_to_learn = 10 # cap on distinct layers to characterize
options.num_randomizations = 20 # random Pauli twirling instances per depth
options.shots_per_randomization = 500 # measurement shots per instance

# --- Submit and retrieve ---
learner = NoiseLearner(mode=backend, options=options)

job_no_dd = learner.run([tqc_no_dd])
job_dd = learner.run([tqc_dd])

noise_no_dd = job_no_dd.result()
noise_dd = job_dd.result()

# --- Extract Pauli Lindblad rates ---
layer_no_dd = noise_no_dd.data[0]
generators = layer_no_dd.error.generators.to_labels()
rates_no_dd = np.array(layer_no_dd.error.rates)

layer_dd = noise_dd.data[0]
rates_dd = np.array(layer_dd.error.rates)
```

Python

van den Berg, E. *et al.*, *Nature Physics* **19**, 1116–1121 (2023)

Discussion and reflection

Part one

We surveyed different types of noise relevant to superconducting qubits - what they are, how they occur, and how to characterize them.

Part two

We discussed the challenges of executing algorithms and circuits on noisy quantum hardware, and reviewed strategies to combat noise at an algorithm level.

Part three

We showed, through an example, how different types of noise affect circuit fidelity, using noisy simulation.

Part four

We showed, through an example, how to characterize the effect of noise on real hardware, using randomized benchmarking, state tomography and the noise learner. We also showed how a simple error suppression technique — dynamical decoupling — can reduce certain types of noise.

The hardware landscape

A brief tour of leading hardware platforms

	How information is encoded	Sources of noise	Strength and challenges
Superconducting qubits	quantized energy levels of nonlinear LC circuits	Dielectric loss, charge/flux noise	Fast gate speeds, scalable fabrication
Trapped ions	hyperfine or optical electronic states of atomic ions (e.g., $^{40}\text{Ca}^+$, $^{171}\text{Yb}^+$) confined by electromagnetic traps	Motional heating, laser noise	Long coherence times, all-to-all connectivity
Neutral atoms	hyperfine ground states of neutral atoms (e.g., ^{87}Rb , ^{133}Cs) held in optical tweezers	Atom loss, Rydberg decay	Highly reconfigurable qubit arrays, long coherence times
Photonics	discrete or continuous-variable (squeezed states) degrees of freedom, e.g., polarization	Photon loss	Room-temperature operation possible, long coherence time